Лабораторна робота №1

Тема: Система управління лікарнею

Група: ТТП-31

Виконала: Бережна Ольга Олександрівна

Розділ 1. Проєктування та реалізація бази даних «Система управління лікарнею»

1.1. Концептуальна модель та аналіз сутностей (ER-діаграма)

На початковому етапі проєктування було розроблено концептуальну модель бази даних, яка повністю покриває предметну область медичного закладу та містить **понад 15 основних сутностей**. Це забезпечує повноцінний облік пацієнтів, персоналу, палат, медикаментів та фінансових операцій.

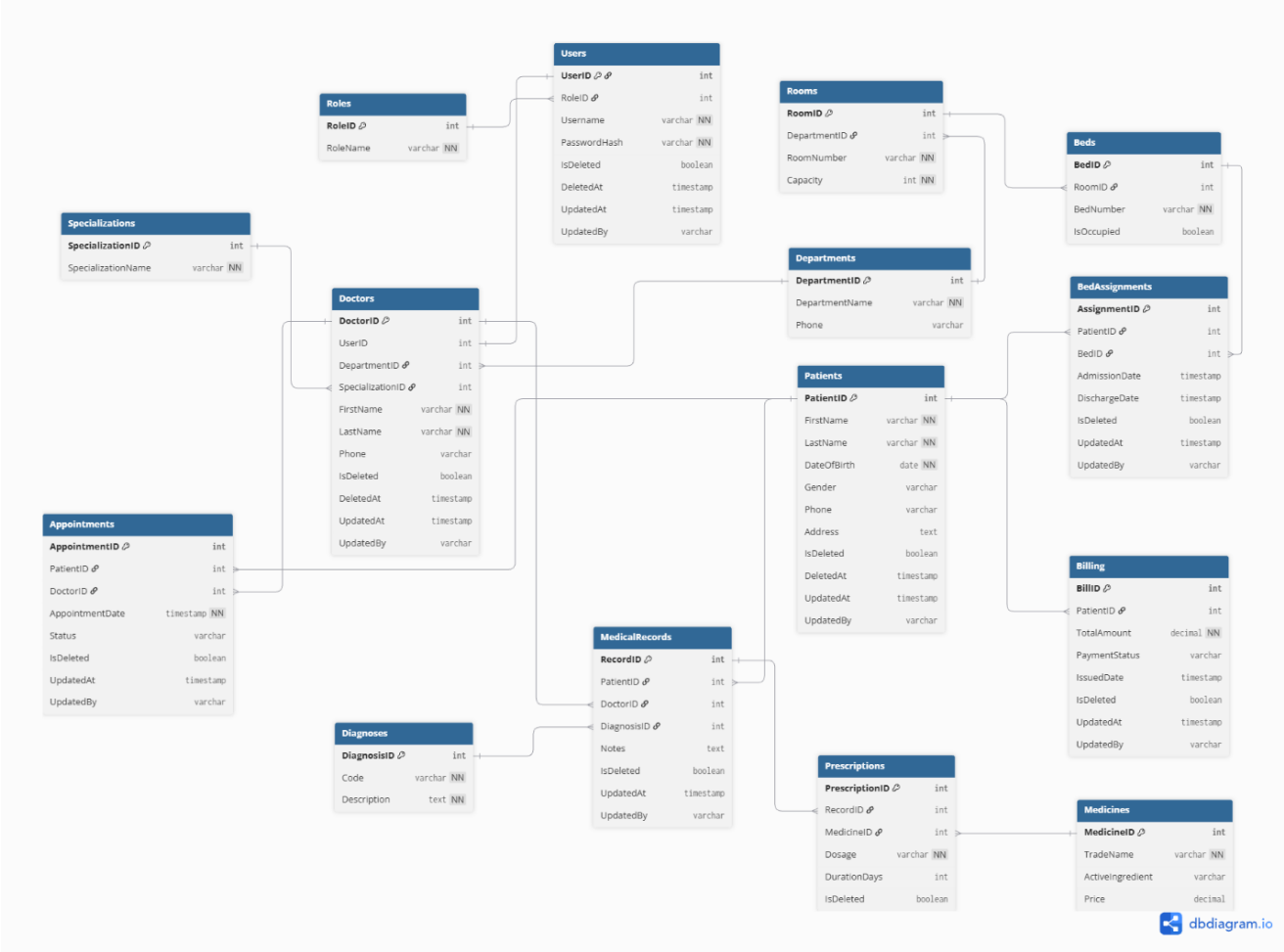


Рис 1. *Схема бази даних «Система управління лікарнею» (ER-діаграма)*

Перелік основних сутностей системи:

1. **Roles** — ролі користувачів у системі (наприклад, адміністратор, лікар, реєстратор).

2. **Users** — облікові записи користувачів системи з полями безпеки (`PasswordHash`), станом (`IsDeleted`), датою оновлення (`UpdatedAt`) та інформацією про те, хто змінив (`UpdatedBy`).

3. **Specializations** — медичні спеціалізації лікарів.

4. **Doctors** — дані про лікарів (пов'язані з `Users, Departments, Specializations`, а також мають підтримку Soft Delete та аудиту).

5. **Departments** — медичні відділення лікарні.

6. **Rooms** — палати у відділеннях із зазначенням місткості (`Capacity`).

7. **Beds** — ліжко-місця в палатах із прапорцем зайнятості (`IsOccupied`).

8. **Patients** — дані про пацієнтів (ПІБ, дата народження, телефон, адреса) із підтримкою Soft Delete та полях аудиту (`UpdatedAt, UpdatedBy`).

9. **BedAssignments** — історія госпіталізації та призначення пацієнтів на конкретні ліжка з датами госпіталізації/виписки (`AdmissionDate, DischargeDate`).

10. **Appointments** — записи пацієнтів на прийом до лікарів із прив'язкою до часу (`AppointmentDate`) та статусу.

11. **Diagnoses** — довідник діагнозів (код та опис).

12. **MedicalRecords** — медичні карти / історії хвороб, що об'єднують пацієнта, лікаря та діагноз.

13. **Medicines** — каталог медикаментів та їхня вартість.

14. **Prescriptions** — призначення ліків за медичними записами (`MedicalRecords`) із зазначенням дозування та тривалості.

15. **Billing** — фінансова інформація та рахунки пацієнтів (`TotalAmount, PaymentStatus`).

1.2 Особливості проєкту на основі діаграми:

Реалізація Soft Delete та аудиту: Такі таблиці, як `Users`, `Doctors`, `Patients`, `BedAssignments`, `Appointments`, `MedicalRecords` та `Billing`, містять поля `IsDeleted`,

`DeletedAt`, `UpdatedAt`, `UpdatedBy`, що повністю задовольняє вимоги безпеки та збереження історії змін.

Зв'язки (Foreign Keys): На діаграмі чітко показано зв'язки типу один-до-багатьох (1-to-Many) та багато-до-багатьох (Many-to-Many через проміжні таблиці, `BedAssignments` та `Prescriptions`).

1.3 Фізична модель у реляційній СУБД PostgreSQL

На основі розробленої концепції було створено фізичну базу даних у СУБД з використанням стандартних типів даних (`integer`, `character varying`, `timestamp without time zone`, `numeric`, `text`, `serial` для автоінкременту, `boolean`, `decimal` для цін).

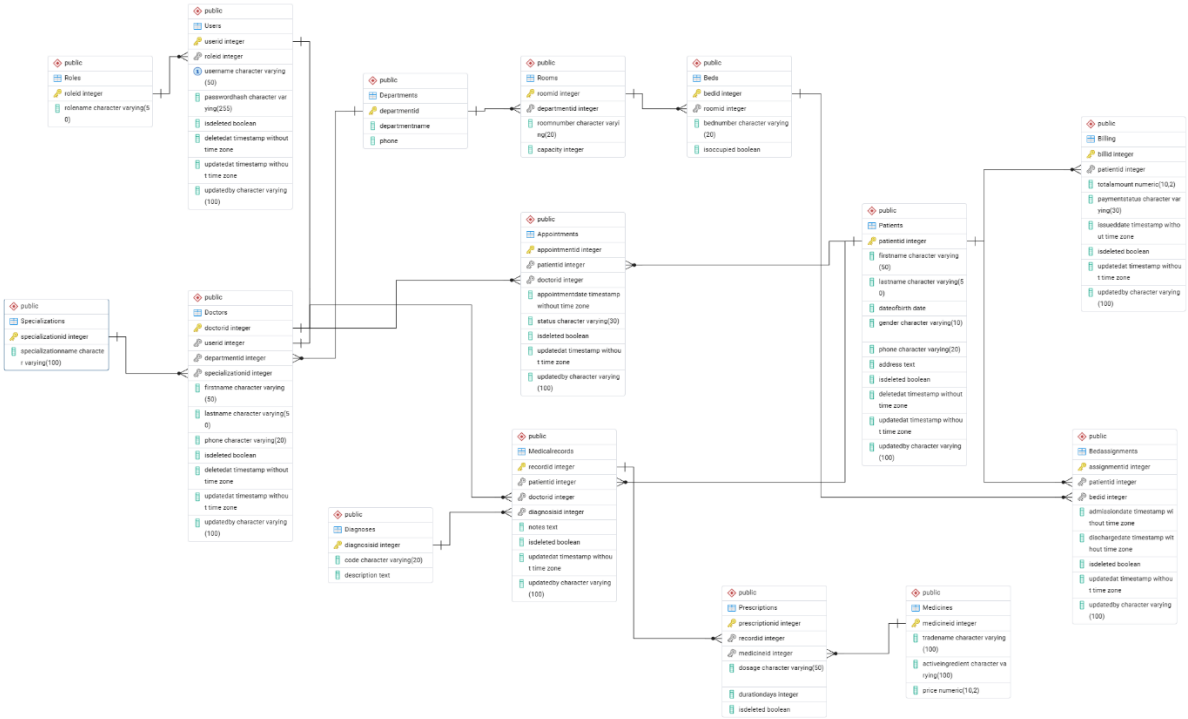


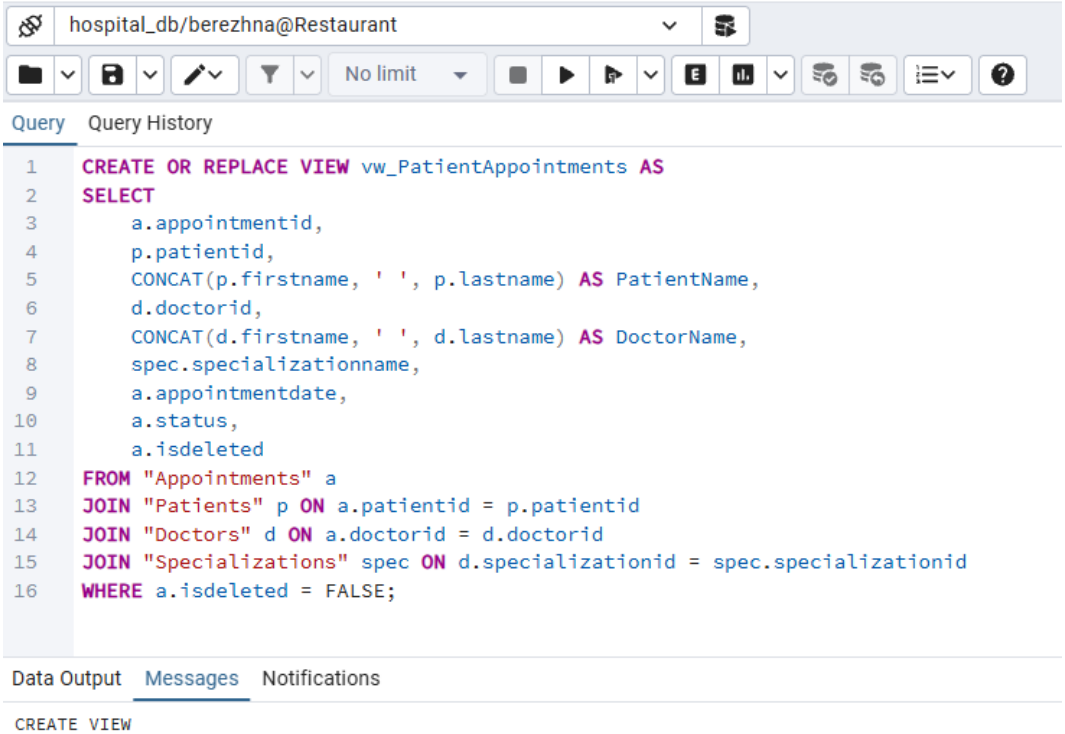
***Рис 2**. Фізична схема реляційної бази даних у СУБД PostgreSQL*

Розділ 2. Програмна реалізація об'єктів бази даних та оптимізація (PostgreSQL)

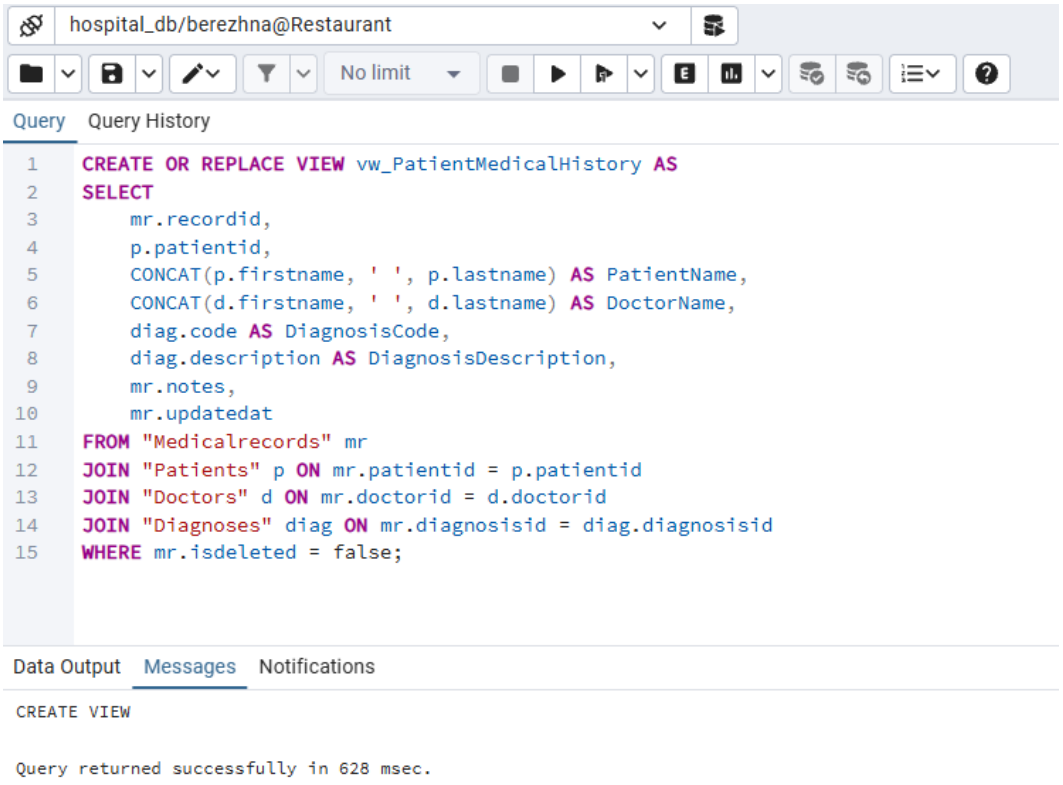
Для забезпечення цілісності даних, автоматизації обліку та безпеки у базі даних PostgreSQL було реалізовано комплекс програмних об'єктів (понад 10 одиниць) і механізмів індексування

2.1. Представлення даних (Views)

1. Представлення детальної інформації про прийоми пацієнтів (`vw_PatientAppointments`):



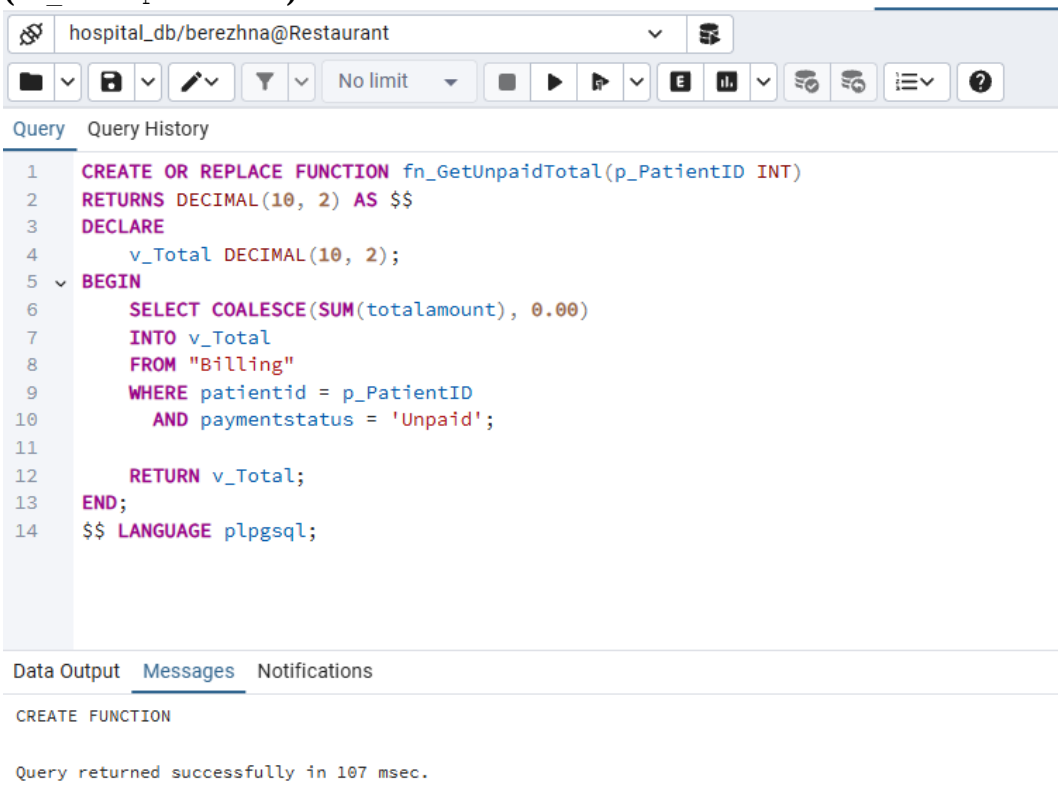
2. Представлення медичної історії пацієнта (`vw_PatientMedicalHistory`):



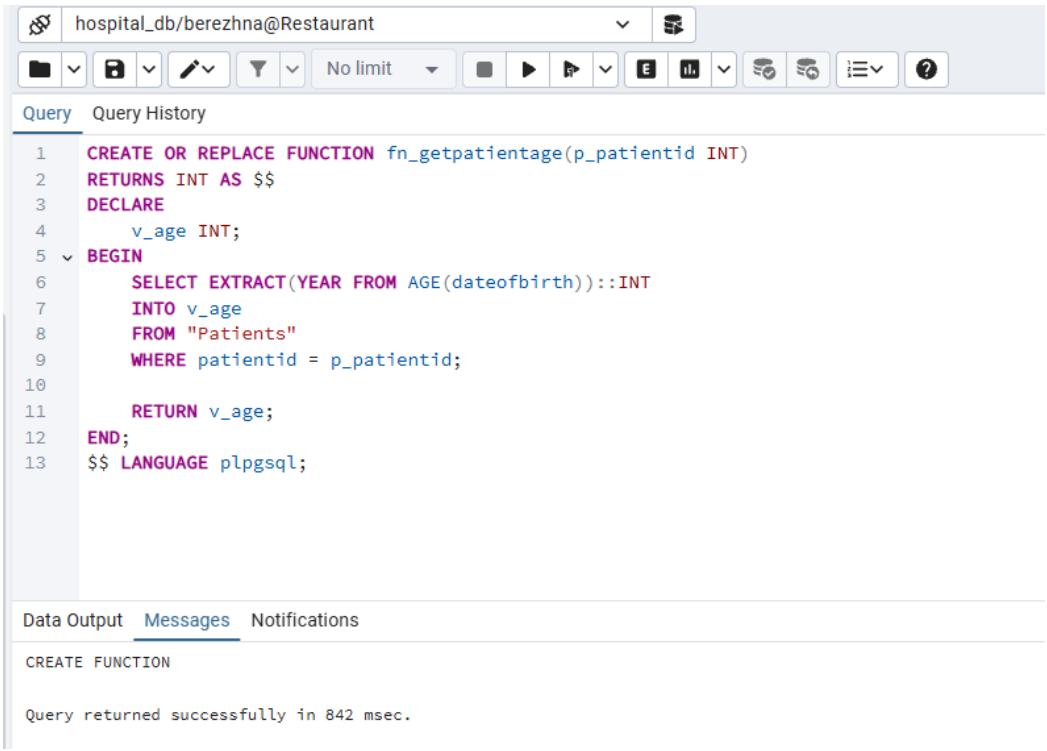
2.2. Користувацькі функції (Functions)

Функції дозволяють інкапсулювати логіку розрахунків та повертати обчислено-агреговані значення.

1. Функція для підрахунку загальної суми несплачених рахунків пацієнта (`fn_GetUnpaidTotal`):



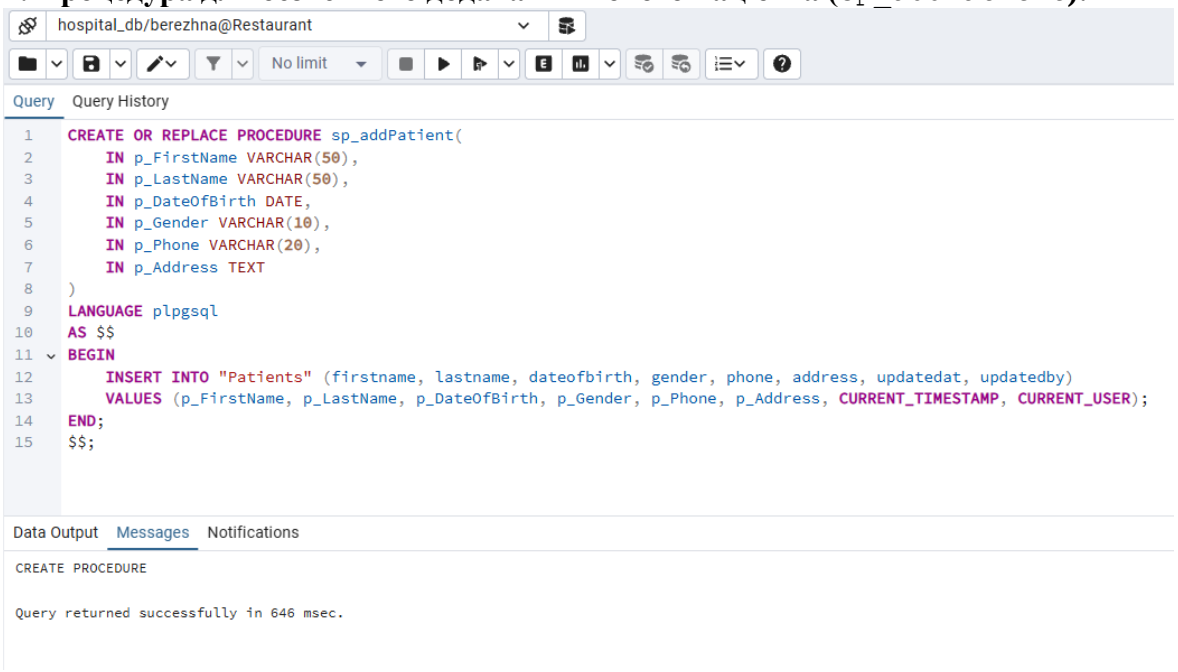
2. Функція розрахунку віку пацієнта (`fn_getpatientage`):



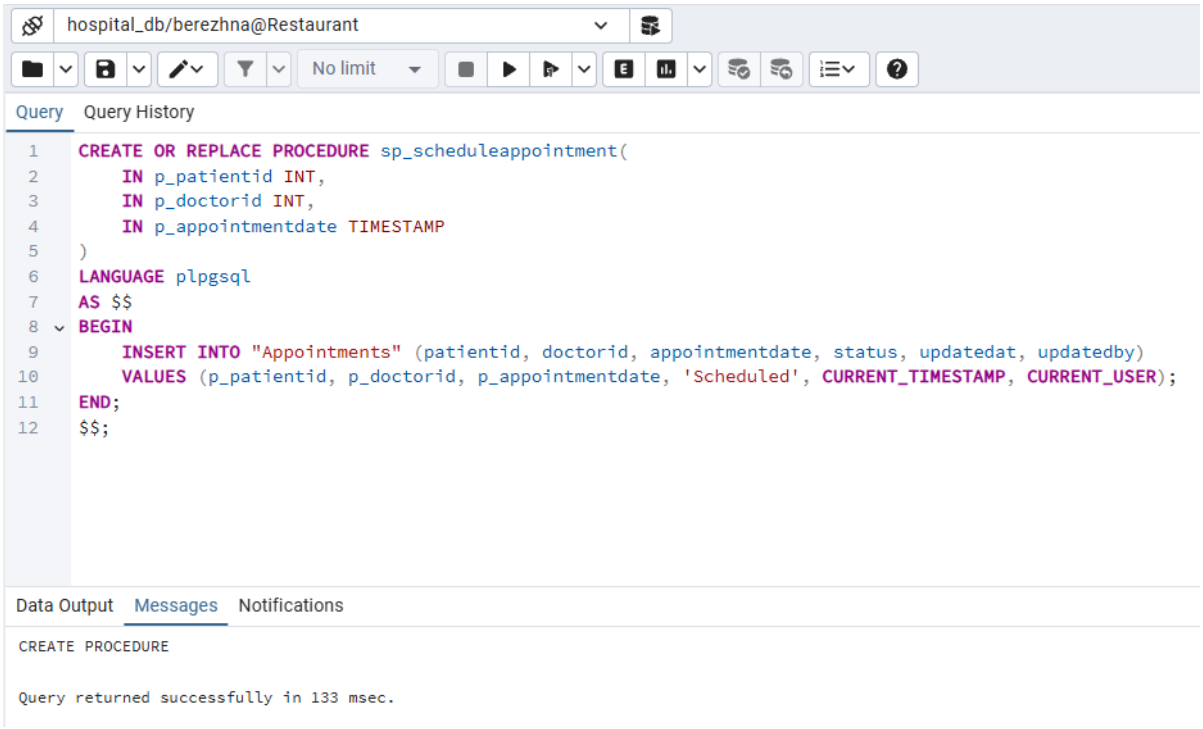
2.3. Збережені процедури (Stored Procedures)

Процедури забезпечують виконання транзакційних операцій зміни даних (створення записів, госпіталізації тощо).

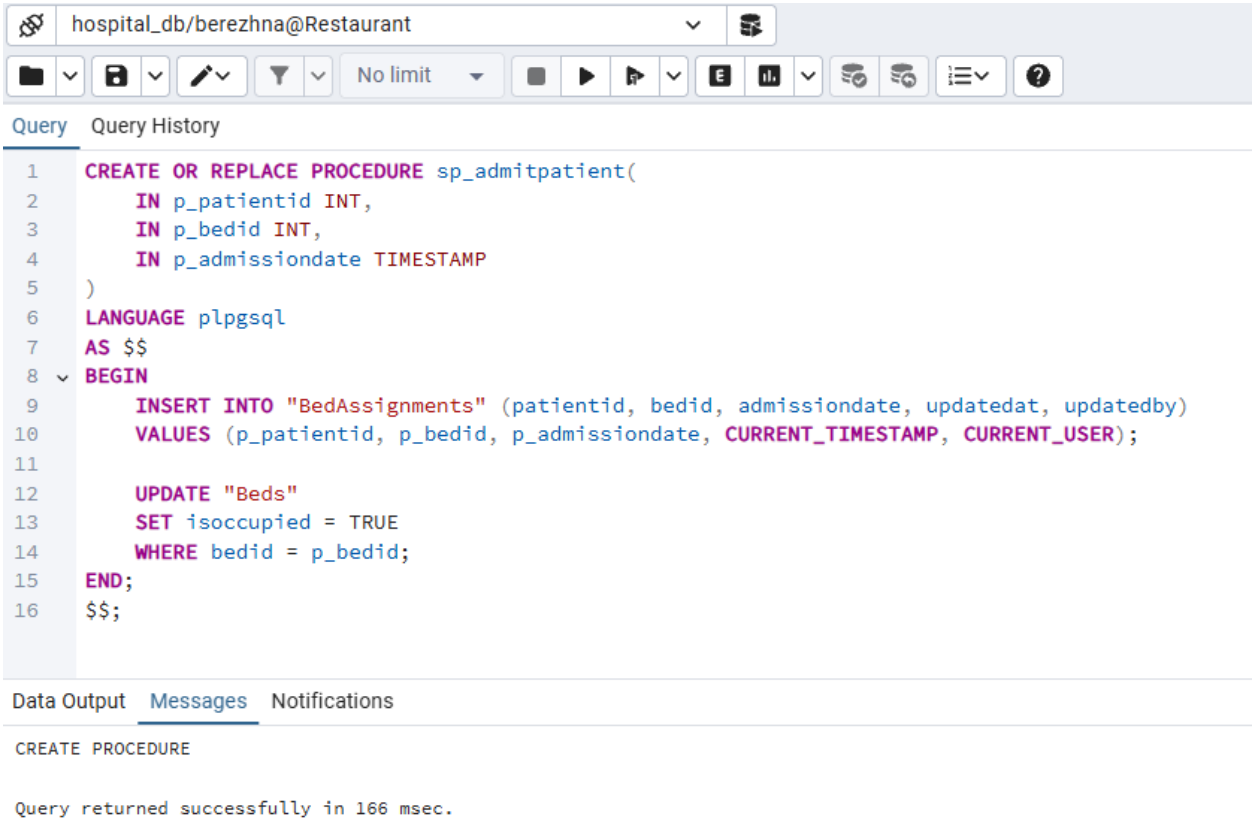
1. Процедура для безпечного додавання нового пацієнта (`sp_addPatient`):



2. Процедура для створення запису на прийом до лікаря (`sp_scheduleappointment`):



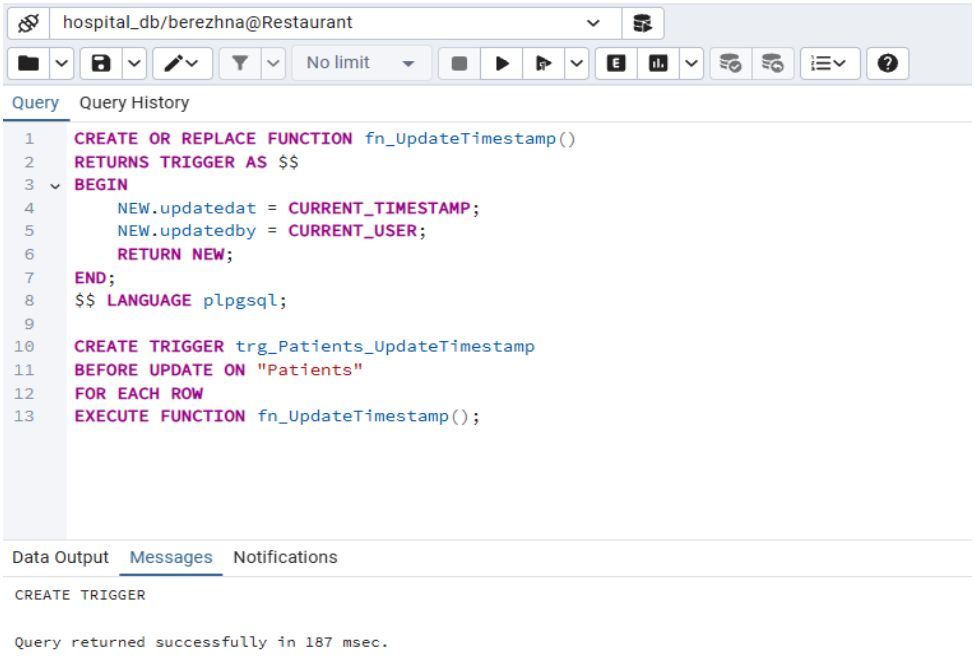
3. Процедура госпіталізації та призначення ліжка (`sp_admitpatient`):



2.4. Тригери та функції-тригери (Triggers)

Тригери автоматично виконуються при виникненні подій `INSERT`, `UPDATE` або `DELETE`, гарантуючи підтримку аудиту та консистентність даних.

1. Тригер автоматичного оновлення міток часу (`trg_Patients_UpdateTimestamp`):



2.Тригер управління зайнятістю ліжок (`trg_BedAssignments_Occupancy`):

```
CREATE OR REPLACE FUNCTION fn_UpdateBedOccupancy()
RETURNS TRIGGER AS $$
BEGIN

   IF (TG_OP = 'INSERT') THEN
      UPDATE "Beds"
      SET isoccupied = TRUE
      WHERE bedid = NEW.bedid;

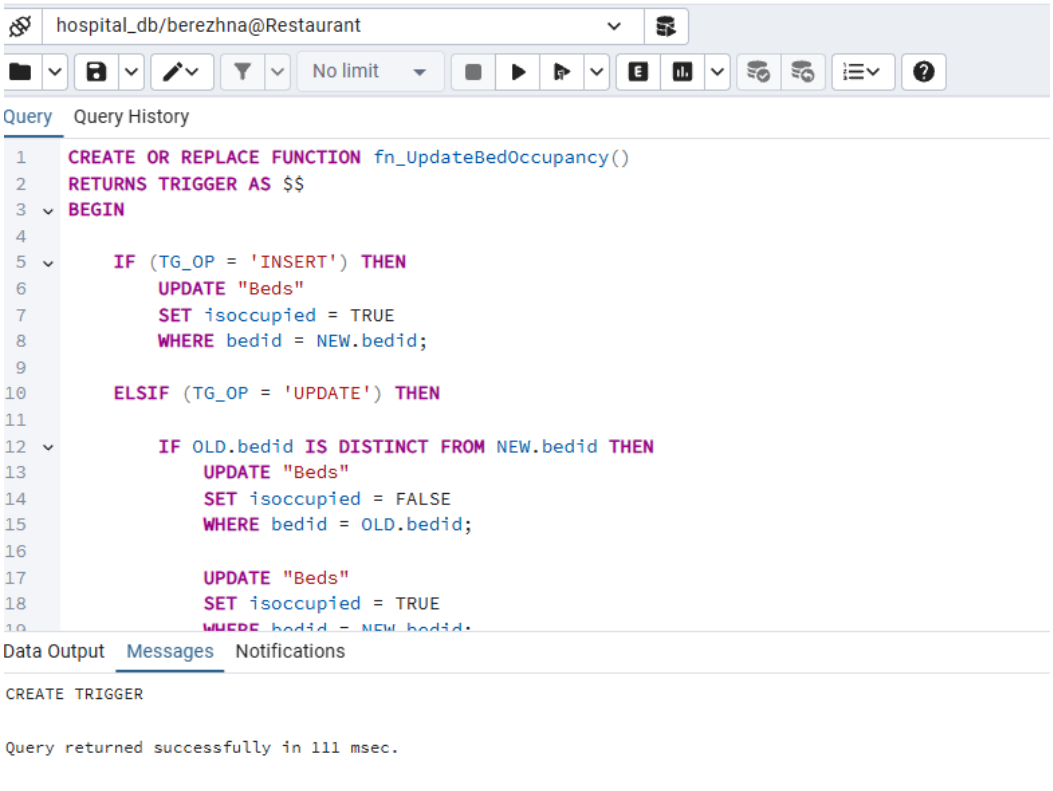
   ELSIF (TG_OP = 'UPDATE') THEN

      IF OLD.bedid IS DISTINCT FROM NEW.bedid THEN
         UPDATE "Beds"
         SET isoccupied = FALSE
         WHERE bedid = OLD.bedid;

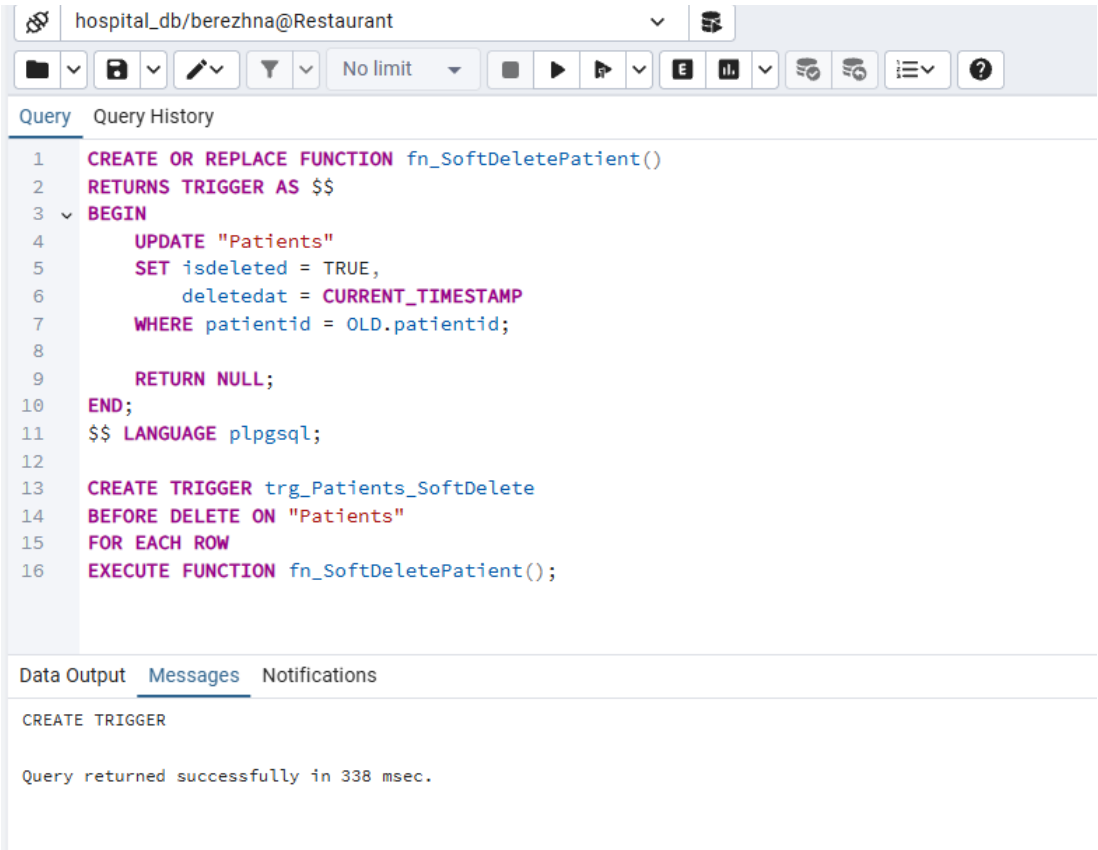
         UPDATE "Beds"
         SET isoccupied = TRUE
         WHERE bedid = NEW.bedid;
      END IF;
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_BedAssignments_Occupancy
AFTER INSERT OR UPDATE ON "Bedassignments"
FOR EACH ROW
EXECUTE FUNCTION fn_UpdateBedOccupancy();
```



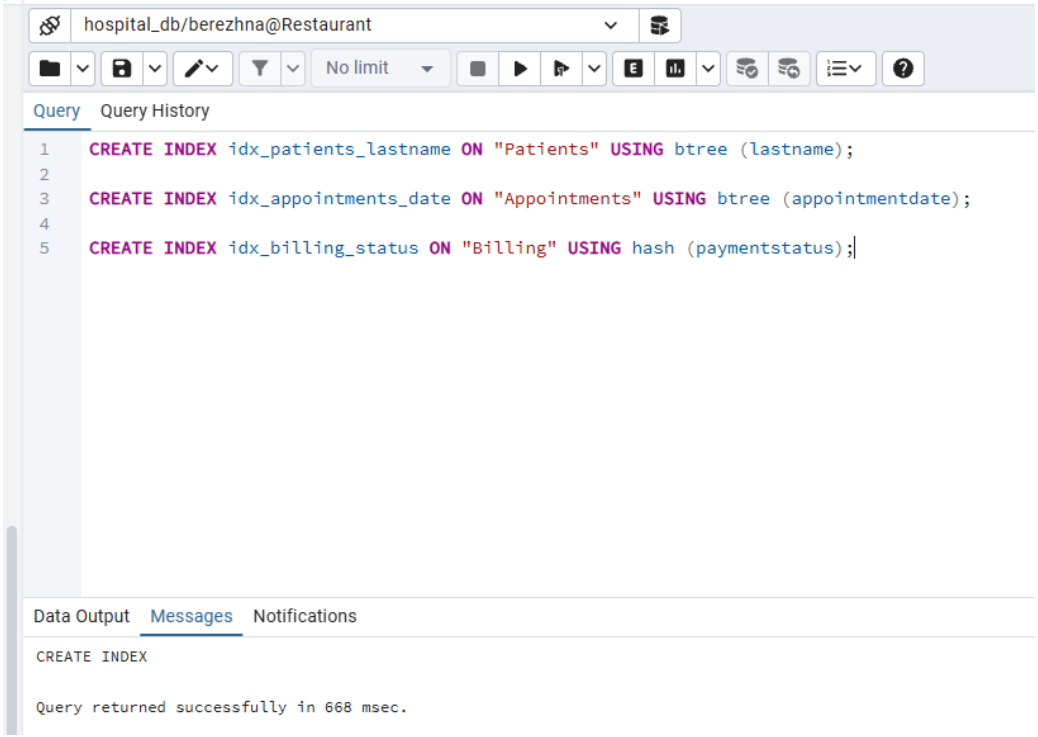
3.Тригер «М'якого видалення» (`trg_Patients_SoftDelete`):



2.5. Оптимізація та індексація бази даних

Для забезпечення швидкого пошуку інформації та мінімізації часу виконання запитів у системі було створено індекси **двох різних типів**:

- **B-Tree індекси (**`idx_patients_lastname, idx_appointments_date`**):** Використовуються за замовчуванням у PostgreSQL. Вони організовують дані у вигляді збалансованого дерева, що дозволяє базі знайти потрібного пацієнта чи дату за мізерну кількість кроків (замість того, щоб перебирати всю таблицю рядок за рядком — так званий *Sequential Scan*).
- **Hash-індекс (**`idx_billing_status`**):** Створюється за допомогою алгоритму хешування. Він ідеально оптимізований для точної перевірки рівності (`WHERE paymentstatus = 'Unpaid'`), забезпечуючи миттєвий доступ до фінансових звітів.



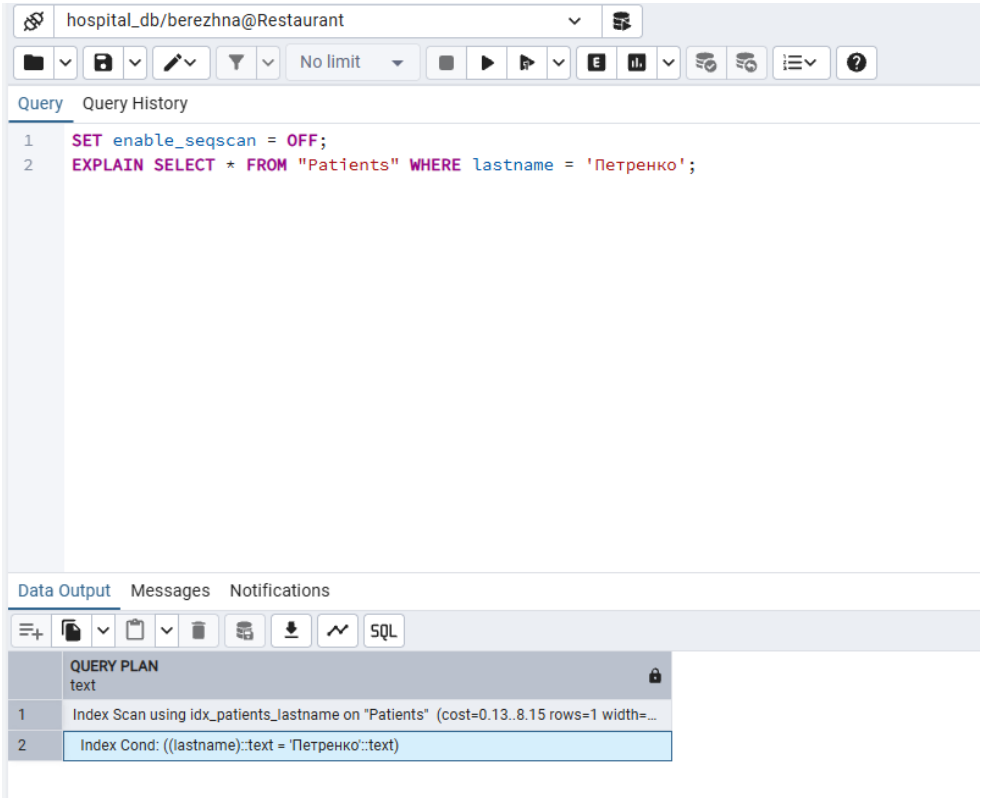


Рисунок 3 *. Перевірка плану виконання запиту (`EXPLAIN`) та використання B-Tree індексу за прізвищем для таблиці `Patients`*

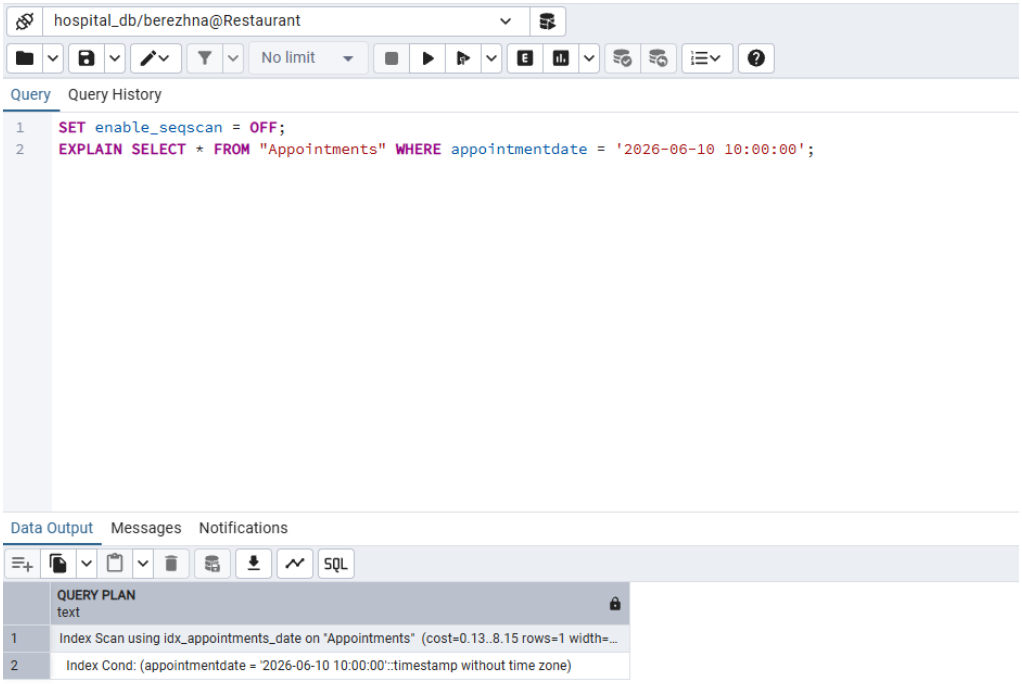


Рисунок 4 *. Перевірка плану виконання запиту (`EXPLAIN`) для B-Tree індексу за полем дати прийому в таблиці `Appointments`*

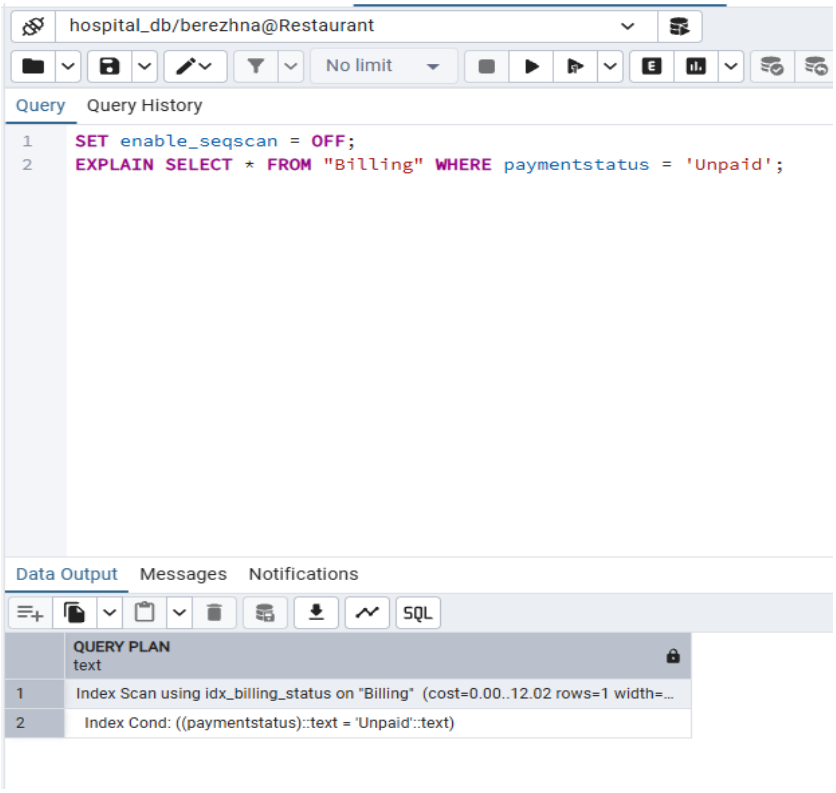


Рисунок 5 *. Перевірка плану виконання запиту (`EXPLAIN`) для Hash-індексу за полем статусу оплати в таблиці `Billing`*

Розділ 3. Програмна архітектура та робота з кодом (C#)

Для взаємодії з базою даних у додатку на C# реалізовано шаблони проєктування:

Repository Pattern: Інкапсулює логіку доступу до даних для конкретних сутностей (`PatientRepository`, `AppointmentRepository`), приховуючи прямі SQL-запити.

Unit of Work Pattern: Керує життєвим циклом транзакції (`IDbTransaction`). Забезпечує принцип ACID: усі операції (наприклад, додавання пацієнта + виклик процедури госпіталізації) виконуються в межах однієї транзакції. Успіх завершується методом `uow.Commit()`, а при помилці спрацьовує відкат `uow.Rollback()`.

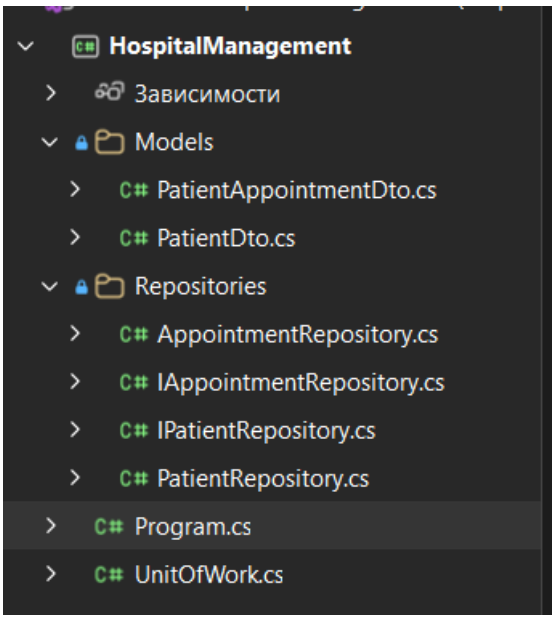


Рисунок 6 *. Структура файлів і папок програмного додатку*

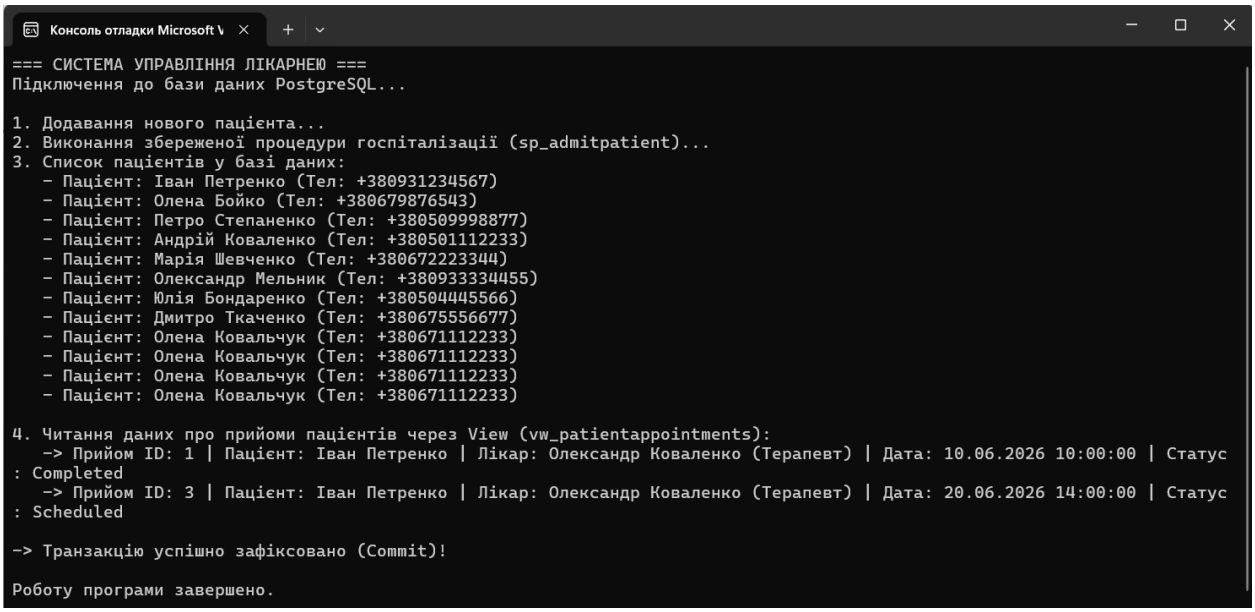


Рисунок 7*. Консольний вивід результатів виконання програми (додавання пацієнта, виконання процедури госпіталізації, зчитування даних через представлення vw_patientappointments та успішне зафіксування транзакції)*